

# **LK-01: Inisialisasi Proyek MLOps**

-

**Machine Learning Operations (MLOps)**

**Kelas A**

**Perancangan Sistem Predictive Autoscaling Berbasis Machine Learning pada Aplikasi Jastip dan Preloved**

**Nama:**

Oktavianus Samuel Minarto

**Dosen:**

Rizal Setya Perdana, S.Kom., M.Kom., Ph.D.



**Program Studi Teknik Informatika  
Universitas Brawijaya  
2026**

## DAFTAR ISI

|                                                                                                                  |           |
|------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Daftar Isi.....</b>                                                                                           | <b>2</b>  |
| <b>1 Identifikasi Masalah dan Domain.....</b>                                                                    | <b>4</b>  |
| 1.1 Latar Belakang dan Permasalahan.....                                                                         | 4         |
| Gambar 1.1. Arsitektur aplikasi Jastip dan Preloved yang digunakan sebagai workload eksperimen.....              | 4         |
| Gambar 1.2. Perbandingan konsep reactive autoscaling dan predictive autoscaling.....                             | 5         |
| 1.2 Machine Learning Task.....                                                                                   | 6         |
| Gambar 1.3. Konsep Machine Learning untuk prediksi workload.....                                                 | 6         |
| <b>2 Data Dinamis dan Data Drift.....</b>                                                                        | <b>8</b>  |
| 2.1 Sumber dan Karakteristik Data.....                                                                           | 8         |
| Gambar 2.1. Alur pengumpulan data workload aplikasi melalui Prometheus.....                                      | 8         |
| Tabel 1.1. Matriks Identifikasi Masalah dan Program/Kegiatan.....                                                | 8         |
| Tabel 1.2. Contoh Data yang Dikumpulkan.....                                                                     | 9         |
| 2.2 Perubahan Workload dan Data Drift.....                                                                       | 9         |
| Gambar 2.2. Steady workload relatif stabil dari waktu ke waktu.....                                              | 9         |
| Gambar 2.3. Gradual Increase, Jumlah request meningkat secara bertahap.....                                      | 10        |
| Gambar 2.4. Spike, terjadi peningkatan workload secara tiba-tiba.....                                            | 10        |
| Gambar 2.5. Periodic/Bursty, workload mengalami pola berulang atau beberapa lonjakan dalam periode tertentu..... | 10        |
| Gambar 2.6. Hubungan perubahan distribusi data dengan proses Continuous Training.....                            | 12        |
| <b>3 Strategi Continuous Training.....</b>                                                                       | <b>13</b> |
| 3.1 Data Collection dan Training Pipeline.....                                                                   | 13        |
| Gambar 3.1. Rancangan pipeline Continuous Training dari pengumpulan data hingga evaluasi model.....              | 14        |
| 3.2 Retraining dan Model Promotion.....                                                                          | 14        |
| Gambar 3.2. Mekanisme trigger, retraining, evaluasi, dan promotion model.....                                    | 15        |
| <b>4 Arsitektur dan Evaluasi Sistem.....</b>                                                                     | <b>16</b> |
| 4.1 Arsitektur Sistem.....                                                                                       | 16        |
| Gambar 4.1. Rancangan arsitektur sistem MLOps dan predictive autoscaling.....                                    | 16        |
| Tabel 4.1. Komponen utama sistem.....                                                                            | 17        |
| 4.2 Kriteria Keberhasilan.....                                                                                   | 17        |
| Tabel 4.2. Kriteria evaluasi sistem.....                                                                         | 17        |
| Gambar 4.2. Rancangan evaluasi predictive autoscaling terhadap reactive autoscaling.....                         | 19        |
| <b>5 Batasan dan Pengembangan.....</b>                                                                           | <b>20</b> |
| 5.1 Batasan Proyek.....                                                                                          | 20        |
| 5.2 Pengembangan Lanjutan.....                                                                                   | 20        |
| Gambar 5.1. Roadmap pengembangan proyek predictive autoscaling.....                                              | 21        |

# 1 IDENTIFIKASI MASALAH DAN DOMAIN

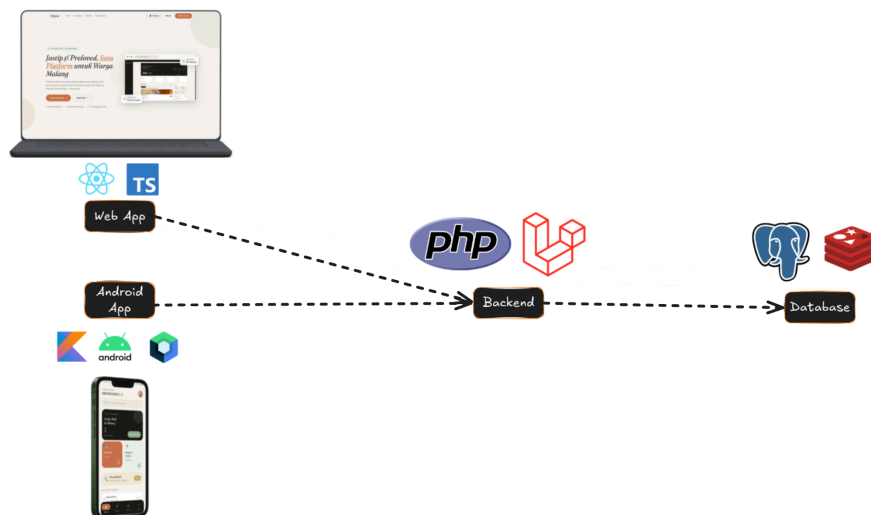
## 1.1 Latar Belakang dan Permasalahan

Aplikasi digital yang melayani banyak pengguna dapat mengalami perubahan beban kerja secara dinamis. Perubahan tersebut dapat terjadi akibat meningkatnya jumlah pengguna, perubahan pola aktivitas, maupun munculnya lonjakan request dalam periode tertentu. Kondisi tersebut dapat menyebabkan kebutuhan terhadap resource komputasi pada aplikasi berubah dari waktu ke waktu.

Pada lingkungan aplikasi yang menggunakan containerization dan orchestration, Kubernetes dapat digunakan untuk melakukan autoscaling, yaitu menyesuaikan jumlah instance aplikasi berdasarkan kondisi workload dan resource yang tersedia. Pendekatan autoscaling yang bersifat reaktif umumnya melakukan scaling setelah kondisi tertentu terjadi, misalnya ketika utilisasi CPU telah melewati suatu threshold. Pendekatan tersebut berpotensi mengalami keterlambatan dalam merespons lonjakan workload karena peningkatan kapasitas baru dilakukan setelah beban meningkat.

Proyek ini menggunakan aplikasi Jastip dan Preloved yang telah dikembangkan sebelumnya sebagai workload eksperimen. Aplikasi tersebut merupakan platform semi-e-commerce yang memungkinkan pengguna untuk membuat dan mencari listing jasa titip (jastip), serta membuat dan mencari penawaran barang preloved.

Aplikasi tersedia dalam dua client, yaitu aplikasi web berbasis React dan aplikasi Android berbasis Kotlin dengan Jetpack Compose. Kedua client tersebut menggunakan satu backend berbasis PHP Laravel sebagai pusat layanan aplikasi.



**Gambar 1.1.** Arsitektur aplikasi Jastip dan Preloved yang digunakan sebagai workload eksperimen.

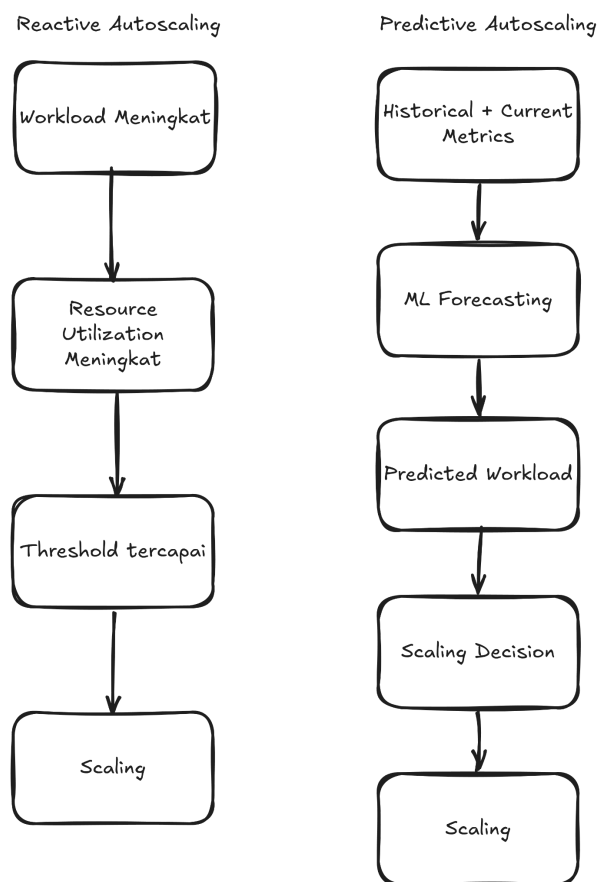
Aktivitas pengguna pada web maupun Android menghasilkan request menuju backend Laravel. Request tersebut kemudian menghasilkan perubahan pada kondisi resource dan performa

sistem, seperti request rate, CPU utilization, memory utilization, response latency, dan jumlah instance backend.

Hal tersebut memungkinkan aplikasi existing digunakan sebagai real workload untuk eksperimen MLOps, tanpa perlu membuat aplikasi khusus hanya untuk menghasilkan data eksperimen.

Berdasarkan kondisi tersebut, proyek ini mengusulkan penerapan predictive autoscaling berbasis Machine Learning. Berbeda dengan autoscaling reaktif yang mengambil keputusan berdasarkan kondisi saat ini, predictive autoscaling mencoba memperkirakan kondisi workload pada periode berikutnya berdasarkan data historis dan kondisi workload terkini.

Secara konseptual, perbedaannya adalah sebagai berikut:



**Gambar 1.2.** Perbandingan konsep reactive autoscaling dan predictive autoscaling.

Dengan pendekatan tersebut, model Machine Learning diharapkan dapat memberikan informasi mengenai kemungkinan perubahan workload sebelum kondisi tersebut benar-benar terjadi. Informasi prediksi selanjutnya dapat digunakan sebagai salah satu input dalam menentukan kebutuhan scaling aplikasi.

Namun, tantangan proyek tidak hanya terletak pada pembuatan model prediksi. Data workload akan terus bertambah dan karakteristiknya dapat berubah seiring perubahan aktivitas

pengguna dan kondisi aplikasi. Oleh karena itu, model yang telah dilatih juga perlu dikelola agar tetap relevan ketika digunakan pada lingkungan production.

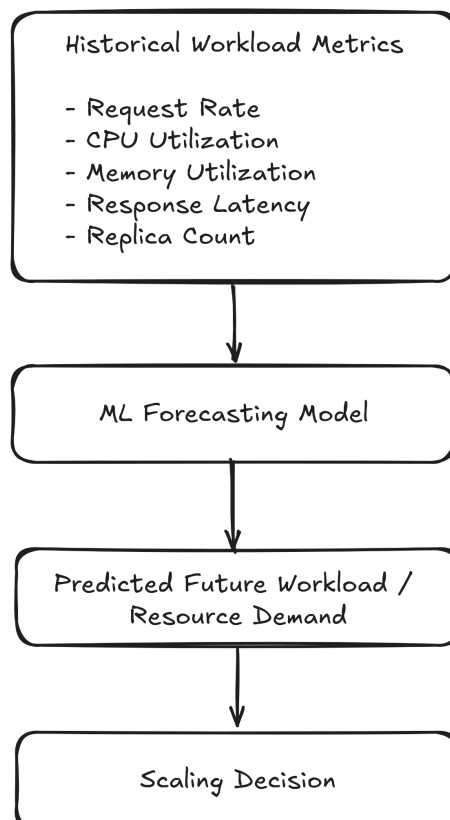
Hal tersebut menjadi alasan utama proyek ini menggunakan pendekatan MLOps, dengan mencakup proses pengumpulan data, training, evaluasi, versioning, deployment, inference, monitoring, dan Continuous Training.

Dengan demikian, permasalahan utama yang ingin diselesaikan dalam proyek ini adalah bagaimana memanfaatkan Machine Learning untuk memprediksi perubahan workload pada aplikasi Jastip dan Preloved sehingga kebutuhan scaling backend dapat diperkirakan secara proaktif, serta bagaimana mengelola lifecycle model agar tetap relevan ketika karakteristik workload berubah dari waktu ke waktu?

## 1.2 Machine Learning Task

Task Machine Learning yang digunakan dalam proyek ini direncanakan berupa Time-Series Forecasting/Regression. Model akan memanfaatkan data historis dan kondisi workload terkini untuk memprediksi kondisi workload pada periode berikutnya.

Secara umum, alur Machine Learning dapat digambarkan sebagai berikut:



**Gambar 1.3.** Konsep Machine Learning untuk prediksi workload.

Beberapa kandidat fitur yang akan digunakan meliputi:

- Request rate, untuk menggambarkan jumlah request yang diterima backend dalam periode tertentu.

- CPU utilization, untuk menggambarkan penggunaan resource CPU.
- Memory utilization, untuk menggambarkan penggunaan memory.
- Response latency, untuk menggambarkan performa respons aplikasi.
- Replica count, untuk mengetahui kapasitas aplikasi yang sedang tersedia.
- Network metrics, apabila relevan berdasarkan hasil observasi data.

Target prediksi belum ditentukan secara final pada tahap awal. Kandidat target meliputi:

1. Future request rate
2. Future CPU utilization
3. Future workload/resource demand

Target akhir akan ditentukan setelah data aktual berhasil dikumpulkan dan dilakukan exploratory data analysis. Pendekatan ini digunakan agar pemilihan target Machine Learning didasarkan pada karakteristik workload yang benar-benar dihasilkan oleh aplikasi, bukan hanya berdasarkan asumsi awal.

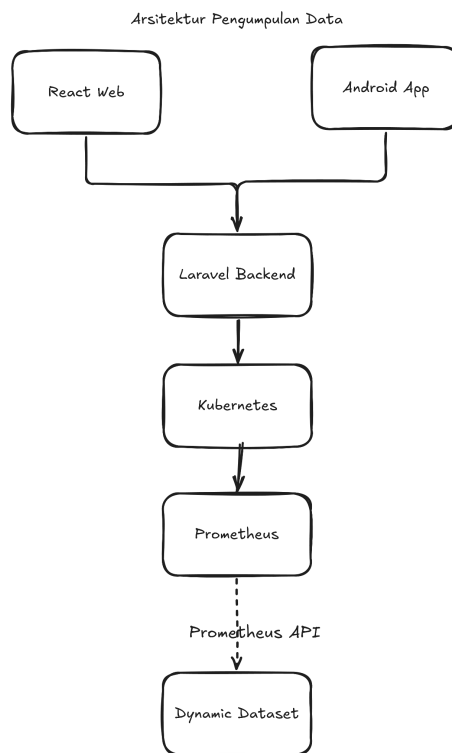
Hasil prediksi nantinya akan menjadi input bagi komponen scaling policy. Model Machine Learning bertugas menghasilkan prediksi, sedangkan scaling policy bertugas menerjemahkan prediksi tersebut menjadi rekomendasi jumlah resource atau replica. Dengan pemisahan tersebut, model Machine Learning tetap dapat dievaluasi secara independen dari mekanisme scaling, sekaligus memungkinkan sistem dikembangkan secara bertahap menuju predictive autoscaling yang terintegrasi dengan Kubernetes.

## 2 DATA DINAMIS DAN DATA DRIFT

### 2.1 Sumber dan Karakteristik Data

Data yang digunakan dalam proyek ini berasal dari operational metrics aplikasi Jastip dan Preloved yang berjalan pada lingkungan Kubernetes. Aplikasi menerima request dari dua client, yaitu web berbasis React dan aplikasi Android berbasis Kotlin Jetpack Compose. Kedua client menggunakan backend Laravel yang sama sehingga aktivitas pengguna dari kedua platform akan menghasilkan workload pada service backend.

Untuk memperoleh data secara berkelanjutan, sistem monitoring menggunakan Prometheus. Prometheus akan mengumpulkan metrics dari infrastruktur dan aplikasi yang berjalan di Kubernetes. Data tersebut kemudian dapat diakses melalui Prometheus API untuk digunakan sebagai sumber dataset Machine Learning.



**Gambar 2.1.** Alur pengumpulan data workload aplikasi melalui Prometheus.

Data yang direncanakan untuk dikumpulkan meliputi:

**Tabel 1.1. Matriks Identifikasi Masalah dan Program/Kegiatan**

| NO. | Data               | Deskripsi                                             |
|-----|--------------------|-------------------------------------------------------|
| 1.  | Timestamp          | Waktu pengamatan metrics                              |
| 2.  | Request Rate       | Jumlah request yang diterima backend per satuan waktu |
| 3.  | CPU Utilization    | Persentase penggunaan CPU                             |
| 4.  | Memory Utilization | Penggunaan memory aplikasi/container                  |
| 5.  | Response Latency   | Waktu yang dibutuhkan backend untuk merespons request |

|    |                 |                                                   |
|----|-----------------|---------------------------------------------------|
| 6. | Replica Count   | Jumlah instance backend yang sedang berjalan      |
| 7. | Network Metrics | Aktivitas jaringan yang berkaitan dengan workload |

Tidak seluruh data tersebut secara otomatis akan digunakan sebagai fitur Machine Learning. Pemilihan fitur akhir akan dilakukan berdasarkan hasil eksplorasi dataset dan hubungan masing-masing metrics terhadap target prediksi.

Data yang dikumpulkan memiliki karakteristik time-series, karena setiap observasi memiliki timestamp dan menggambarkan kondisi sistem pada waktu tertentu. Sebagai contoh:

**Tabel 1.2. Contoh Data yang Dikumpulkan**

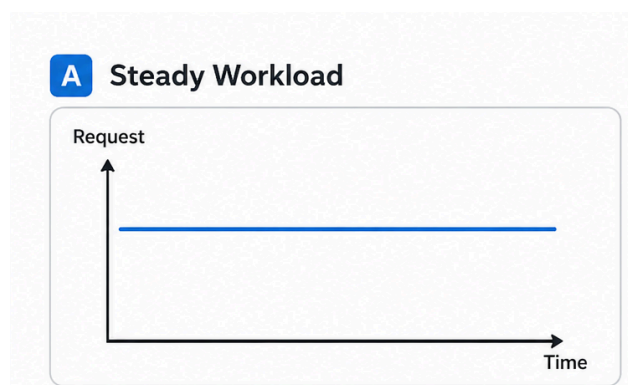
| Waktu | Request Rate | CPU | RAM | Latency | Replica |
|-------|--------------|-----|-----|---------|---------|
| 10:00 | 20 req/s     | 25% | 42% | 80 ms   | 1       |
| 10:01 | 35 req/s     | 32% | 44% | 85 ms   | 1       |
| 10:02 | 60 req/s     | 45% | 47% | 105 ms  | 1       |
| 10:03 | 90 req/s     | 63% | 51% | 145 ms  | 2       |
| 10:04 | 120 req/s    | 77% | 55% | 180 ms  | 2       |

Data akan terus bertambah selama aplikasi menerima workload baru. Dengan demikian, berbeda dengan dataset statis yang dikumpulkan satu kali, dataset pada proyek ini memiliki sifat *Continuously growing and time-dependent data*. Pertambahan data memungkinkan model untuk memperoleh informasi baru mengenai karakteristik workload aplikasi dan menjadi dasar untuk proses retraining pada tahap *Continuous Training*.

## 2.2 Perubahan Workload dan Data Drift

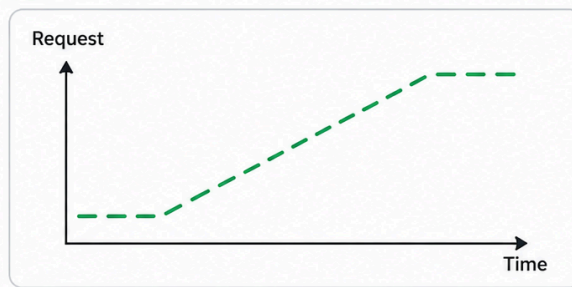
Karakteristik workload aplikasi tidak selalu konstan. Request yang diterima backend dapat berubah berdasarkan jumlah pengguna, jenis aktivitas, waktu penggunaan, maupun kondisi tertentu pada aplikasi.

Sebagai contoh, workload dapat mengalami pola berikut:



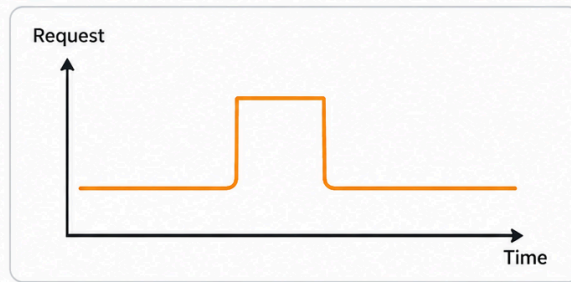
**Gambar 2.2.** Steady workload relatif stabil dari waktu ke waktu

### B Gradual Increase



**Gambar 2.3.** Gradual Increase, Jumlah request meningkat secara bertahap

### C Spike



**Gambar 2.4.** Spike, terjadi peningkatan workload secara tiba-tiba

### D Periodic / Bursty



**Gambar 2.5.** Periodic/Bursty, workload mengalami pola berulang atau beberapa lonjakan dalam periode tertentu

Pola tersebut dapat dihasilkan melalui aktivitas nyata dari web dan Android maupun melalui controlled load testing sehingga eksperimen dapat dilakukan secara terukur dan reproducible.

Perubahan karakteristik workload dapat menyebabkan data drift, yaitu kondisi ketika distribusi data yang digunakan pada saat model digunakan berbeda dengan distribusi data yang digunakan ketika model dilatih.

Sebagai contoh, model pada awalnya dapat dilatih menggunakan data:

*Training Data*

*Request Rate:*

20 – 80 req/s

Kemudian setelah aplikasi digunakan pada kondisi yang berbeda:

*Production Data*

*Request Rate:*

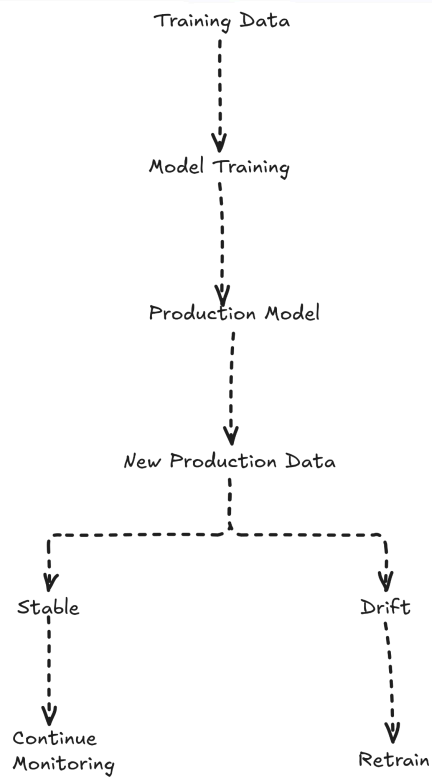
50 – 200 req/s

Perubahan distribusi tersebut dapat menyebabkan model menghasilkan prediction error yang lebih tinggi karena kondisi production tidak lagi sepenuhnya sesuai dengan pola yang dipelajari pada saat training.

Selain perubahan request rate, drift juga dapat muncul pada:

- CPU utilization;
- memory utilization;
- response latency;
- pola request;
- jumlah replica;
- karakteristik waktu penggunaan aplikasi.

Potensi data drift dalam proyek ini penting karena model predictive autoscaling akan digunakan pada data yang terus berkembang. Model yang awalnya memiliki performa baik dapat mengalami penurunan performa apabila karakteristik workload berubah secara signifikan. Oleh karena itu, perubahan data akan menjadi salah satu faktor yang dipertimbangkan dalam Continuous Training. Apabila terdeteksi perubahan distribusi atau penurunan performa model yang signifikan, sistem dapat memicu proses retraining menggunakan data terbaru.



**Gambar 2.6.** Hubungan perubahan distribusi data dengan proses Continuous Training.

Dengan pendekatan tersebut, data dinamis tidak hanya digunakan untuk menambah jumlah training data, tetapi juga menjadi dasar untuk menentukan kapan model perlu diperbarui agar tetap relevan terhadap kondisi workload aplikasi.

## 3 STRATEGI CONTINUOUS TRAINING

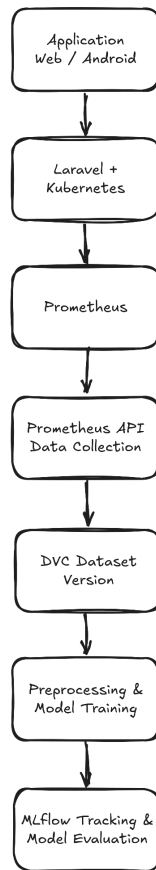
### 3.1 Data Collection dan Training Pipeline

Continuous Training pada proyek ini dirancang untuk menjaga agar model Machine Learning tetap relevan terhadap perubahan workload aplikasi. Proses dimulai dari pengumpulan operational metrics secara berkala melalui Prometheus. Metrics yang diperoleh dari lingkungan Kubernetes, seperti request rate, CPU utilization, memory utilization, response latency, dan replica count, diambil melalui Prometheus API dan disimpan sebagai data time-series. Data baru akan terus ditambahkan selama aplikasi menerima workload, baik dari aktivitas pengguna melalui web dan Android maupun dari controlled load testing.

Data yang telah dikumpulkan tidak langsung digunakan untuk melakukan training. Data terlebih dahulu melalui proses validasi untuk memastikan timestamp, nilai metrics, dan struktur data berada dalam kondisi yang sesuai. Data yang valid kemudian digabungkan dengan dataset sebelumnya dan dilakukan versioning menggunakan DVC. Dataset yang telah diperbarui selanjutnya digunakan dalam proses preprocessing dan training model. Proses eksperimen dan hasil training dicatat menggunakan MLflow, sehingga setiap model dapat dikaitkan dengan dataset, konfigurasi training, serta metrik evaluasinya.

Pipeline tersebut dirancang sebagai proses yang dapat dijalankan secara otomatis sehingga penambahan data baru tidak memerlukan proses manual pada setiap tahapan. Secara umum, alur data collection hingga training dapat digambarkan sebagai berikut.

Continuous Training Pipeline



**Gambar 3.1.** Rancangan pipeline Continuous Training dari pengumpulan data hingga evaluasi model.

Pipeline tersebut memungkinkan proses training dilakukan menggunakan data terbaru tanpa mengubah model production secara langsung. Hasil training akan menghasilkan candidate model yang selanjutnya masuk ke tahap evaluasi dan model promotion.

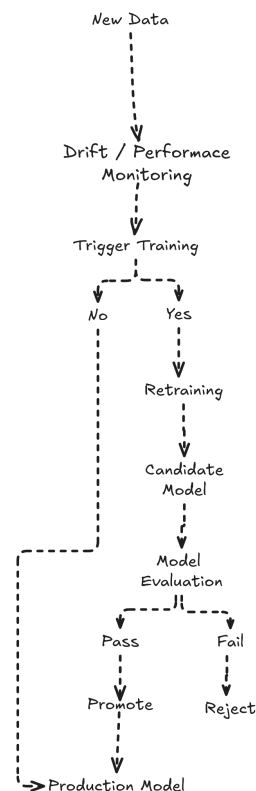
### 3.2 Retraining dan Model Promotion

Retraining dilakukan ketika model perlu diperbarui berdasarkan penambahan data maupun perubahan karakteristik workload. Pada tahap awal, proyek menggunakan dua mekanisme trigger, yaitu scheduled trigger dan condition-based trigger. Scheduled trigger menjalankan proses pemeriksaan secara berkala untuk mengetahui apakah telah tersedia data baru dalam jumlah yang cukup untuk melakukan training ulang. Sementara itu, condition-based trigger digunakan ketika sistem mendeteksi perubahan distribusi data atau penurunan performa model yang melewati batas yang telah ditentukan.

Ketika salah satu kondisi tersebut terpenuhi, data terbaru akan digunakan untuk membentuk dataset training dan proses retraining dijalankan. Model hasil retraining tidak langsung menggantikan model yang sedang digunakan pada production. Model tersebut terlebih dahulu

diperlakukan sebagai candidate model dan dievaluasi menggunakan data pengujian yang sesuai. Hasil evaluasi kemudian dibandingkan dengan performa model production yang sedang digunakan.

Apabila candidate model memenuhi kriteria performa yang telah ditentukan dan memberikan hasil yang lebih baik atau setidaknya memenuhi batas minimum yang ditetapkan, model dapat dipromosikan menjadi production model melalui MLflow Model Registry. Sebaliknya, apabila performanya tidak memenuhi kriteria, candidate model ditolak dan model production sebelumnya tetap digunakan. Mekanisme tersebut bertujuan untuk mencegah proses retraining justru menurunkan performa sistem.



**Gambar 3.2.** Mekanisme trigger, retraining, evaluasi, dan promotion model.

Dengan mekanisme tersebut, Continuous Training tidak hanya berfungsi untuk menghasilkan model baru, tetapi juga memastikan bahwa perubahan model dilakukan secara terkontrol. Setiap versi model dapat ditelusuri berdasarkan dataset dan hasil eksperimennya, sedangkan model production hanya diperbarui ketika candidate model memenuhi kriteria yang telah ditentukan.

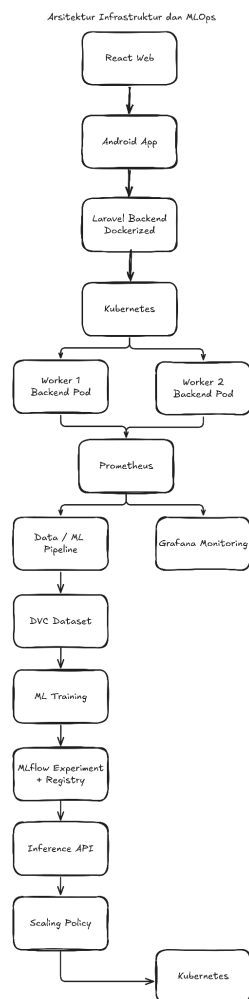
Pada implementasi selanjutnya, mekanisme ini akan menjadi bagian dari closed-loop MLOps, yaitu ketika data production terus dikumpulkan, model dipantau, perubahan kondisi dideteksi, dan proses retraining dapat dipicu kembali menggunakan data terbaru. Dengan demikian, lifecycle model tidak berhenti pada deployment pertama, tetapi terus beradaptasi terhadap perkembangan workload aplikasi.

## 4 ARSITEKTUR DAN EVALUASI SISTEM

### 4.1 Arsitektur Sistem

Sistem dirancang sebagai lingkungan MLOps yang menghubungkan aplikasi existing, infrastructure Kubernetes, monitoring, serta Machine Learning pipeline. Aplikasi Jastip dan Preloved digunakan sebagai workload utama, sedangkan backend Laravel menjadi service yang diamati dan menjadi target utama eksperimen autoscaling. Client web berbasis React dan aplikasi Android berbasis Kotlin Jetpack Compose menghasilkan request menuju backend sehingga membentuk workload yang dapat diamati secara langsung.

Infrastructure eksperimen direncanakan menggunakan tiga virtual machine pada lingkungan lab. Satu VM digunakan sebagai control plane sekaligus menampung komponen monitoring, sedangkan dua VM lainnya digunakan sebagai worker untuk menjalankan workload aplikasi. Pembagian ini memungkinkan jumlah replica backend didistribusikan pada beberapa node sehingga perubahan kebutuhan resource dapat diamati selama eksperimen.



**Gambar 4.1.** Rancangan arsitektur sistem MLOps dan predictive autoscaling.

Komponen utama yang digunakan dalam rancangan tersebut dirangkum pada Tabel 4.1.

**Tabel 4.1. Komponen utama sistem**

| Komponen            | Teknologi                | Fungsi                                       |
|---------------------|--------------------------|----------------------------------------------|
| Web Client          | React                    | Menghasilkan workload dari sisi web          |
| Mobile Client       | Kotlin + Jetpack Compose | Menghasilkan workload dari sisi mobile       |
| Backend             | Laravel/PHP              | Service utama yang menjadi target eksperimen |
| Containerization    | Docker                   | Menjalankan aplikasi dalam container         |
| Orchestration       | Kubernetes               | Mengelola deployment dan scaling backend     |
| Monitoring          | Prometheus               | Mengumpulkan operational metrics             |
| Visualization       | Grafana                  | Memvisualisasikan kondisi sistem             |
| Dateset Versioning  | DVC                      | Mengelola versi dataset                      |
| Experiment Tracking | MLflow                   | Mencatat eksperimen dan performa model       |
| Model Registry      | MLflow                   | Mengelola versi model                        |
| Inference           | FastAPI                  | Menyediakan model untuk prediction           |
| CI/CD               | Github Actions           | Mengotomatisasi proses pipeline MLOps        |

Arsitektur tersebut dirancang agar lifecycle Machine Learning dapat berjalan secara terintegrasi. Data dari production environment dikumpulkan melalui monitoring, digunakan dalam proses training, kemudian model yang telah dievaluasi dapat digunakan melalui inference API. Hasil prediksi selanjutnya dapat digunakan oleh scaling policy sebagai dasar pengambilan keputusan scaling.

Dengan rancangan tersebut, proses MLOps tidak berhenti pada deployment model, tetapi membentuk siklus yang menghubungkan data collection, training, deployment, inference, monitoring, dan Continuous Training.

## 4.2 Kriteria Keberhasilan

Evaluasi proyek tidak hanya dilakukan berdasarkan akurasi model Machine Learning, tetapi juga berdasarkan kemampuan sistem dalam mendukung kebutuhan autoscaling. Pendekatan ini diperlukan karena model dengan prediction error rendah belum tentu menghasilkan scaling behavior yang baik pada lingkungan production.

Kriteria evaluasi dibagi menjadi tiga kelompok, yaitu model metrics, system metrics, dan operational metrics.

**Tabel 4.2. Kriteria evaluasi sistem**

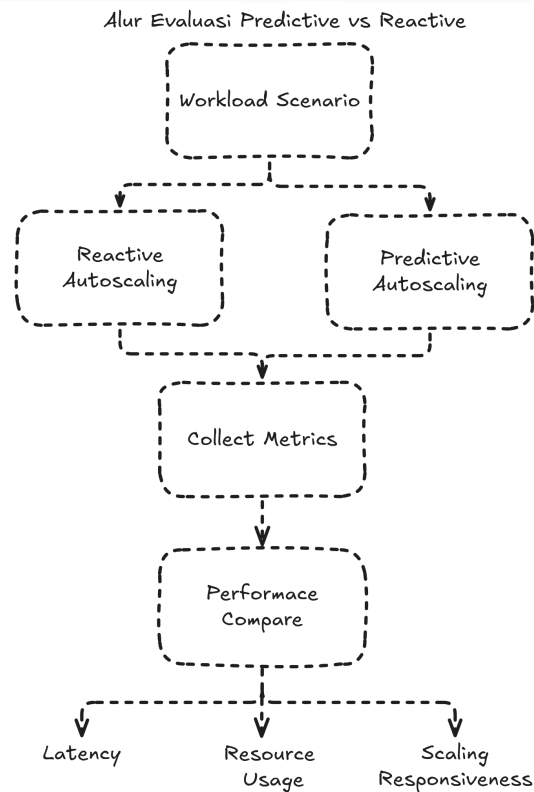
| Kelompok | Metrik                    | Tujuan Evaluasi                       |
|----------|---------------------------|---------------------------------------|
| Model    | Mean Absolute Error (MAE) | Mengukur rata-rata kesalahan prediksi |

|             |                                       |                                                                 |
|-------------|---------------------------------------|-----------------------------------------------------------------|
| Model       | Root Mean Square Error (RMSE)         | Mengukur kesalahan prediksi dengan penalti terhadap error besar |
| Model       | Mean Absolute Percentage Error (MAPE) | Mengukur kesalahan prediksi secara relatif                      |
| System      | Inference Latency                     | Mengukur waktu yang dibutuhkan model menghasilkan prediksi      |
| System      | Response Latency                      | Mengamati dampak workload terhadap respons aplikasi             |
| System      | CPU & Memory Utilization              | Mengukur penggunaan resource                                    |
| System      | Replica Count                         | Mengamati perubahan kapasitas backend                           |
| Operational | Scaling Responsiveness                | Mengukur kemampuan sistem merespons perubahan workload          |
| Operational | Resource Efficiency                   | Mengukur efisiensi penggunaan resource                          |
| Operational | Reliability                           | Mengamati kemampuan sistem mempertahankan layanan               |

Untuk model Machine Learning, MAE, RMSE, dan MAPE digunakan sebagai metrik utama karena task yang digunakan berupa forecasting/regression. Target awal yang digunakan adalah memperoleh prediction error yang cukup rendah untuk memberikan informasi workload yang berguna bagi scaling policy. Nilai threshold final akan ditentukan setelah dataset aktual tersedia sehingga target evaluasi tidak ditetapkan berdasarkan asumsi terhadap workload yang belum diamati.

Pada tingkat sistem, evaluasi dilakukan dengan mengamati hubungan antara perubahan workload, resource utilization, response latency, dan jumlah replica. Pengujian dilakukan menggunakan beberapa skenario workload sehingga dapat diketahui bagaimana sistem merespons kondisi workload yang stabil, meningkat secara bertahap, maupun mengalami spike.

Pada tingkat operasional, keberhasilan proyek ditinjau dari kemampuan predictive autoscaling dalam mengurangi keterlambatan scaling dan penggunaan resource yang tidak diperlukan. Sistem juga akan dibandingkan dengan pendekatan reactive autoscaling sebagai baseline untuk mengetahui apakah penggunaan prediksi memberikan keuntungan yang terukur.



**Gambar 4.2.** Rancangan evaluasi predictive autoscaling terhadap reactive autoscaling.

Perbandingan tersebut menjadi dasar untuk menentukan apakah pendekatan predictive autoscaling memberikan manfaat yang nyata terhadap sistem. Hasil evaluasi juga akan digunakan sebagai masukan untuk pengembangan Continuous Training, karena perubahan performa model maupun perubahan karakteristik workload dapat menjadi indikator bahwa model perlu diperbarui.

Dengan demikian, keberhasilan proyek tidak hanya ditentukan oleh kemampuan model dalam melakukan prediksi, tetapi oleh kemampuan keseluruhan pipeline MLOps dalam menghasilkan, mengelola, menyajikan, dan mempertahankan model yang dapat digunakan secara reliable dalam lingkungan aplikasi yang workload-nya berubah secara dinamis.

## **5 BATASAN DAN PENGEMBANGAN**

### **5.1 Batasan Proyek**

Proyek ini berfokus pada penerapan Machine Learning dan praktik MLOps untuk mendukung predictive autoscaling pada backend aplikasi Jastip dan Preloved. Aplikasi existing digunakan sebagai workload eksperimen, sedangkan proses pengumpulan data, training, deployment, monitoring, dan Continuous Training dirancang sebagai bagian dari pipeline MLOps.

Untuk menjaga agar proyek dapat diselesaikan sesuai ruang lingkup perkuliahan, beberapa batasan diterapkan. Backend Laravel menjadi target utama eksperimen autoscaling, sementara frontend React dan aplikasi Android berperan sebagai client yang menghasilkan workload. Database tidak menjadi target utama autoscaling pada tahap awal. Infrastruktur eksperimen juga dibatasi pada lingkungan tiga VM yang disediakan oleh laboratorium.

Pada sisi Machine Learning, proyek menggunakan pendekatan forecasting/regression dan tidak berfokus pada pengembangan algoritma Machine Learning baru. Target prediksi, fitur, prediction horizon, serta model terbaik akan ditentukan berdasarkan hasil eksplorasi terhadap data aktual. Oleh karena itu, keputusan teknis tersebut belum dikunci pada tahap perancangan awal.

Selain itu, proyek tidak mengasumsikan bahwa predictive autoscaling selalu lebih baik daripada reactive autoscaling. Keunggulan pendekatan predictive akan ditentukan melalui eksperimen menggunakan workload yang terkontrol dan metrik evaluasi yang telah ditentukan pada bab sebelumnya.

### **5.2 Pengembangan Lanjutan**

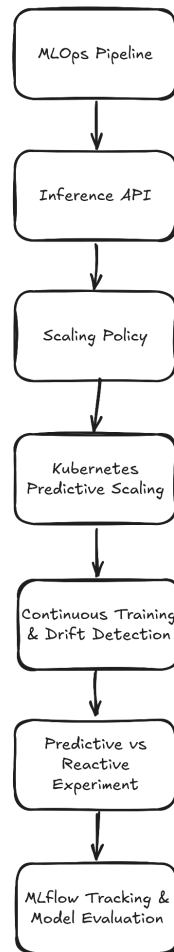
Setelah pipeline dasar MLOps berhasil dibangun, proyek dapat dikembangkan menjadi sistem predictive autoscaling yang lebih terintegrasi dengan Kubernetes. Pengembangan dimulai dengan menghubungkan inference API dengan scaling policy sehingga hasil prediksi workload dapat digunakan untuk memberikan rekomendasi jumlah replica backend.

Tahap selanjutnya adalah melakukan integrasi scaling policy dengan Kubernetes sehingga proses scaling dapat dilakukan secara otomatis. Sistem kemudian dapat diuji menggunakan berbagai pola workload, seperti steady, gradual increase, spike, periodic, dan bursty. Pengujian tersebut memungkinkan dilakukan perbandingan antara predictive autoscaling dan reactive autoscaling menggunakan kondisi eksperimen yang sama.

Pada sisi MLOps, pengembangan dapat dilanjutkan dengan memperkuat mekanisme monitoring model, data drift detection, Continuous Training, model promotion, serta rollback. Dengan demikian, model tidak hanya dapat digunakan untuk melakukan prediksi, tetapi juga dapat dikelola secara berkelanjutan ketika karakteristik workload aplikasi berubah.

Dalam jangka lebih panjang, sistem ini dapat menjadi fondasi penelitian mengenai predictive autoscaling pada Kubernetes, khususnya untuk menganalisis hubungan antara forecasting workload, efisiensi resource, responsiveness scaling, dan reliability aplikasi.

Roadmap Pengembangan



**Gambar 5.1.** Roadmap pengembangan proyek predictive autoscaling..

Dengan pengembangan tersebut, proyek MLOps ini tidak hanya menghasilkan prototype pipeline Machine Learning, tetapi juga menyediakan dasar yang dapat dikembangkan menjadi eksperimen dan penelitian yang lebih mendalam mengenai reliable, adaptive, dan resource-efficient autoscaling pada lingkungan cloud-native.